XMLSchéma Description

(Norme du W3C, 2001)

- Langage conçu pour :
  - Offrir une plus grande puissance d'expression que les DTD
  - Être auto-descriptif et exprimé en XML
  - Être simple d'utilisation
  - Être coordonné avec les autres spécifications du W3C
- Permet de définir la structure et le typage des contenus des documents
  - Types simples : integer, float, string, enum, date, etc.
  - Possibilité de types définis par extension de types existants
  - Notion « orienté-*objet* » pour le typage : spécialisation

Définition
Eléments et Attributs

- Elements et attributs sont nommés, typés, contraints
  <xsd:element name=.. type=.. minOccurs=.. maxOccurs=.. >
  <xsd:attribute name=.. type=.. >

Exemple:

```
<xsd:element name="Nom" type="xsd:string">
        <xsd:attribute name="Sexe"  type="xsd:string"/>
</xsd:element>
```

→ <nom sexe="M"> Don </nom>

"Type" : type simple ou type complexe. Les types…

Types simples *dans XSD*

- string
- integer (integer, positiveInteger, negativeInteger, long, short, etc.)
- decimal, float (réels)
- boolean
- time, timeInstant (durée)
- date
- anyURI
- Pour compatibilité avec les DTD
  - NMTOKEN (lettres, chiffres, '.', '-', '_', ':'), NMTOKENS (séparateur : espaces)
  - ID, IDREF, IDREFS (liste d'IDREF avec espaces): identifiants, références
  - ENTITY

Construction de types (1)

- Décrire un type <xsd:complexType> par composition d'éléments et d'attributs
  - Liste ordonnée : sequence
  - Collection non ordonnée : all
  - Alternative : choice

Construction de types (1bis)

Exemple déclaration de type complexe

```
<xsd:complexType name="AddresseFR">
    <xsd:sequence>
        <xsd:element name="nom" type="xsd:string"/>
        <xsd:element name="rue" type="xsd:string"/>
        <xsd:element name="ville" type="xsd:string"/>
        <xsd:element name="codep" type="xsd:decimal"/>
    </xsd:sequence>
    <xsd:attribute name="pays" type="xsd:NMTOKEN" fixed="FR"/>
</xsd:complexType>
```


Construction de types (1bis)

Utilisation du type complexe

```
<xsd:complexType name="AddresseFR">
    <xsd:sequence>
        <xsd:element name="nom" type="xsd:string"/>
        <xsd:element name="rue" type="xsd:string"/>
        <xsd:element name="ville" type="xsd:string"/>
        <xsd:element name="codep" type="xsd:decimal"/>
    </xsd:sequence>
    <xsd:attribute name="pays" type="xsd:NMTOKEN" fixed="FR"/>
</xsd:complexType>
```

<xsd:element name="AdresseEnFrance" type="AddresseFR" >

Construction de types (1bis)

Création d'un élément utilisant le type complexe

```
<xsd:complexType name="AddresseFR">
    <xsd:sequence>
        <xsd:element name="nom" type="xsd:string"/>
        <xsd:element name="rue" type="xsd:string"/>
        <xsd:element name="ville" type="xsd:string"/>
        <xsd:element name="codep" type="xsd:decimal"/>
    </xsd:sequence>
    <xsd:attribute name="pays" type="xsd:NMTOKEN" fixed="FR"/>
</xsd:complexType>
<xsd:element name="AdresseEnFrance" type=" AddresseFR" >

<AdresseEnFrance pays="FR"> <nom>Maret </nom> <rue>…
```


Constructeur de Types (1)

Composition d'éléments par alternative : choice

```
<xsd:complexType name="TemporalPoint">
    <xsd:choice>
        <xsd:element name='year' type='gYear'/>
        <xsd:element name='date' type='date'/>
    </xsd:choice>
</xsd:complexType>

<xsd:element name="when" type="TemporalPoint" >

<when> <year>1998</year> </when>
<when> <date>01/01/2019</date> </when>
```


Constructeurs de Types (2)

- Spécialiser un type simple prédéfini
  - par extension (ajout d'information), avec <xsd:extension base="…">
  - par restriction (ajout de contrainte) avec <xsd:restriction base="…">

Constructeurs de Types (2bis)

Spécialisation de type par extension

(Les étudiants sont des personnes et ont un numéro d'étudiant)

```
<xsd:complexType name="etudiant">
        <xsd:extension base="personne">
                <xsd:sequence>
                    <xsd:element name="numEtu" type="string" />
                    <xsd:element name="StatusCrous" type="…"/>
                </xsd :sequence>
        </xsd :extension>
</xsd :complexType>
```

On suppose que ce type existe

Constructeurs de Types (2ter)

Spécialisation de type par restriction

```
<xsd :simpleType name="entiers2digits">
    <xsd :restriction base="xs:positiveInteger">
        <xsd:pattern value="\d{2}"/>
    </xsd :restriction>
</xsd :simpleType>
```

Valeur sous la forme : 43

```
<xsd:simpleType name="code">
    <xsd:restriction base="xsd:string">
        <xsd:pattern value="[A-Za-z]{5}-\d{2}"/>
    </xsd:restriction>
</xsd:simpleType>
```

Valeur sous la forme : Catch-22

Type Enuméré et type Liste

```
<xsd:simpleType name="Region">
    <xsd:restriction base="xsd:string">
        <xsd:enumeration value="Alsace "/>
        <xsd:enumeration value="Normandie "/>
        <xsd:enumeration value="Occitanie "/> …
    </xsd:restriction>
</xsd:simpleType>
```

Type énuméré

```
<xsd:simpleType name="ListeRegion">
    <xsd:list itemType="Region"/>
</xsd:simpleType>
```

Type liste

```
<xsd:element name="regions" type="ListeRegion"/>
```

Elément

Dans le document : <regions>Alsace Normandie Bretagne</regions>

Réutilisation de déclarations

- Possibilité de référencer un élément
  <xsd:element ref="NomElement" />
  « Importe » l'élément et son type (à la place de name=… type=…)

  Exemple :

```
<xs:element name="note">
  <xs:complexType>     <xs:sequence>
      <xs:element ref="to"/>
      <xs:element ref="from"/>
      <xs:element ref="body"/>
    </xs:sequence>   </xs:complexType>
</xs:element>

<xs:element name="to" type="xs:string"/>
<xs:element name="from" type="xs:string"/>
<xs:element name="body" type="xs:string"/>
```


Réutilisation de déclarations (2)

Réutiliser des schémas définis ailleurs

- Importer un schéma
  <import nameSpace = "http:// ..."
  schemaLocation = "http:// ...../file.xsd" />
  Permet d'utiliser des composants du schéma
- Inclure un schéma
  <include schemaLocation = "http:// ...../file.xsd" />
  Permet d'ajouter tous les composants d'un schéma
- Possibilité d'étendre un schéma
  <redefine schemaLocation="http:// ..."/>
  .... Extensions ...
  </redefine>

Règles d'écriture pour DTD, XSD

- Modularité
  - définir dans des types séparés les parties réutilisables
- Précédence
  - Regrouper les déclarations de types en tête
- Abstraction
  - Utiliser des entités ou types
- Spécificité
  - Éviter les types trop généraux (fourre-tout)
- Simplicité
  - Découper les Schémas ou DTDs trop complexes

Définir en Elément ou en Attribut?
Choix de conception

Exemple
<xsd:attribute name="pays" type="xsd:NMTOKEN" fixed="FR"/>

Pays: définir comme un attribut ou un élément?

En général : Utiliser les attributs pour

- Exprimer des données pré-*codées*
  telle que <addresseFR pays="FR"> … </ addresseFR >
- Exprimer des relations entre éléments (e.g. référence, lien hypertexte)

```
<author id="oc">
        <firstname>Olivier</firstname>
        <lastname>Corby</lastname>
</author>
<paper ref="oc"/>
```

- Compléter la valeur d'un élément (par ex avec une unité)
  <amount currency='euro'>3.14</amount>

Limites de XSchema

- Document de spécifications complexe (plusieurs centaines de pages)
- Limite du pouvoir d'expression
  - (toujours) pas de contrainte de type sur les IDREFs
- Pb de cohérence avec XML
  - n'impose pas la présence d'une racine unique dans un document (≠ XML)
- Nécessite des outils de génération

Programmer XML – exemples de liens

- **Validation de fichier XML**

https://stackoverflow.com/questions/15732/whats-the-best-way-to-validate-an-xml-file-against-an-xsd-file

- **API Java de validation XML**

https://www.ibm.com/developerworks/xml/library/x-javaxmlvalidapi/index.html

Plus d'info…

API de validation d'un XML



Un fichier XML doit être valide vis-à-vis d'une DTD ou d'un XSD. Principe du programme:

1. Une instance de la classe *SchemaFactory* lit le fichier de schema (fichier XSD) et crée un *Schema object*.
2. Le *Schema object* crée un *Validator object*.
3. Le *Validator object* valide un document XML representé par *Source*.

Source : https://www.ibm.com/developerworks/xml/library/x-javaxmlvalidapi/index.html

The Java XML Validation API



```
import java.io.*;
import javax.xml.transform.Source;


import javax.xml.transform.stream.StreamSource;
import javax.xml.validation.*;
import org.xml.sax.SAXException;

public class DocbookXSDCheck {


   public static void main(String[] args) throws SAXException,
IOException {

      // 1. Lookup a factory for the W3C XML Schema language
      SchemaFactory factory =
         SchemaFactory.newInstance("http://www.w3.org/2001/XMLSch
ema");

      // 2. Compile the schema.
      // Here the schema is loaded from a java.io.File, but you could use
      // a java.net.URL or a javax.xml.transform.Source instead.
      File schemaLocation = new
File("/opt/xml/docbook/xsd/docbook.xsd");
      Schema schema = factory.newSchema(schemaLocation);

      // 3. Get a validator from the schema.
      Validator validator = schema.newValidator();

      // 4. Parse the document you want to check.
      Source source = new StreamSource(args[0]);

      // 5. Check the document
      try {
         validator.validate(source);
         System.out.println(args[0] + " is valid.");
      }
      catch (SAXException ex) {
         System.out.println(args[0] + " is not valid because ");
         System.out.println(ex.getMessage());
      }

   }
```


Bilan DTD, XSchéma



- Définir une grammaire formelle des doc.
- DTD et XML Schema Description (XSD)
- DTD : simple, peu riche, limite sur les types
- XSD : complexe mais rigoureux
- La conception de DTD/Schéma se fait à la main ou avec des outils